

Take-Home Assignment — Senior Software Engineer (Growth & Platform)

Background

About Cubby Beds

Cubby Beds is a Denver-based startup that builds smart safety beds for children and adults with complex cognitive conditions like Autism, Epilepsy, Cerebral Palsy, and Dementia. We've served over 20,000 families and are scaling rapidly across three channels: Consumer (direct-to-family), Partner DME (durable medical equipment suppliers), and Payer (insurance/Medicaid).

About This Role

The Senior Software Engineer (Growth & Platform) sits at the intersection of engineering, product thinking, and go-to-market systems. You'll build both internal tools and customer/partner-facing infrastructure, and stand up the data and engineering foundations underneath them. You'll identify high-leverage opportunities, ship solutions, and measure their impact.

What This Assessment Evaluates

This is a realistic preview of the work you'd do at Cubby, and a look at how you think and build. We're evaluating five things:

- **Engineering execution** — Clean, well-structured, production-shaped code you'd be comfortable handing to a teammate. Not a polished demo over messy internals.
- **Data responsibility** — Thoughtful handling of PHI-like data and coverage information in a regulated environment.
- **Product judgment** — Choosing a genuinely high-leverage problem and scoping it well under a time limit.
- **AI application** — Using AI thoughtfully, with real guardrails where it touches partners or patients.

- **Communication** — Explaining what you built, why, and the tradeoffs you made.

There are no trick questions. We want your specific judgment and choices, not a comprehensive survey of everything you could have done.

About This Assignment

One of Cubby's key growth channels is our network of DME (durable medical equipment) supplier partners. Approximately 95% of families get their Cubby bed through insurance, and DME suppliers are the ones who make that happen. Cubby sells to DMEs, who then work with payers to get the bed covered for the family. In a very real way, DME partners are our primary customer. Cubby works with 100+ DME partners today, and this channel is a high-leverage revenue multiplier: every partner we activate and enable well can drive dozens of orders per year. The challenge is that partner engagement varies widely. Some partners are highly active, while others sign up but never place their first order. Getting partners from "signed up" to "confidently selling Cubby" faster is one of our biggest growth levers.

To support them, we maintain a Supplier Portal, a section of our website where DME partners can access onboarding materials, product information, ordering resources, forms, FAQs, and more.

Today, the portal is functional but difficult to navigate. It's structured as a series of static pages with links to other pages, and DME partners often struggle to find what they need quickly. As Cubby scales its partner network, improving this experience is a meaningful opportunity.

Your task is to evaluate the current Supplier Portal, identify opportunities for improvement, and build a working prototype of one of those opportunities.

The DME Supplier Partner

To complete this assignment, it helps to understand who uses the Supplier Portal and what they're trying to do. Here's a representative profile:

Sarah — DME Account Manager

Sarah works at a mid-size durable medical equipment company. Her job is to help families get the equipment they need, covered by insurance. On any given day, she might be:

- **Evaluating a new product** — A family has asked about Cubby beds. Sarah needs to quickly understand the product, pricing, and whether it's a fit for their patient.
- **Preparing an insurance submission** — Sarah needs to pull together the right forms, a Letter of Medical Necessity, supporting research, and state-specific Medicaid requirements.
- **Placing or tracking an order** — Sarah needs to find the order form, understand policies, and know where to go if something needs to change.
- **Answering a question from a colleague or family** — Sarah gets asked things like "What's the weight limit?" or "Is this covered in Florida?" and needs to find answers fast.
- **Onboarding as a new Cubby partner** — Sarah's company just signed on. She needs to understand how to get started, what resources are available, and who to contact.

Sarah is busy, often juggling 10+ patients across multiple medical device manufacturers. She doesn't have time to click through multiple pages to find a single form. She values speed, clarity, and confidence that she has the right information.

Instructions (4–6 hours)

Part 1: Written Pitch (~30 minutes)

Explore the current Cubby Supplier Portal:

<https://cubbybeds.com/pages/supplier-portal-overview>

Navigate through the portal as if you were Sarah. Then write a short pitch that covers:

- **Challenges (4–6 bullets)** — What challenges do you see Sarah encountering?
- **Opportunities (2–4 bullets)** — What improvements do you see? One to two sentences each. These could be UX improvements, new features, AI-powered tools, workflow changes, or anything else.
- **Your recommendation (short paragraph)** — Which single opportunity would you choose to prototype, and why? What makes it the highest-leverage improvement for both the DME partner and for Cubby? Include a rough business impact estimate, tied to a conversion or partner-activation metric. Directional math is fine; show your reasoning.

- **Argue against yourself (2–3 sentences)** — Make the strongest case against your own choice. Which other opportunity is the most compelling alternative, and what would have to be true for it to be the better bet than the one you picked?

Part 2: Working Prototype (~3.5–4.5 hours)

Build a functional prototype of the opportunity you recommended in Part 1.

Build one vertical slice, end-to-end. Four to six hours goes by fast. We would much rather see a single flow that is well structured, tested where it matters, and handles data responsibly than several half-finished features. Depth over breadth. Pick the one path that best demonstrates the experience you're proposing and make it real from the UI down to the data. In your README, name one thing you deliberately chose not to build in this slice, and why. Good scoping under a time limit is part of what we're evaluating.

Requirements:

- It should be a working UI, not a wireframe, mockup, or slide deck.
- Use a real web stack (e.g. React/TypeScript with an API layer, or an equivalent framework you're strong in). We want to see how you structure an application, not just how it looks.
- Submit a **GitHub repository with commit history**. We read the repo: structure, naming, how you handle configuration and secrets, and how the commits show your thinking. A single squashed commit tells us nothing about how you work.
- You are responsible for sourcing or mocking whatever data your prototype needs. The portal is public, so feel free to pull content from it, create realistic sample data, or both.
- The prototype doesn't need to be production-ready, but it should be production-shaped: something a teammate could read, run, and extend.
- **Tests:** Include tests for the part of your prototype you'd be most nervous shipping. We are not grading coverage percentage. We want to see your judgment about what deserves a test, and whether the test actually protects the behavior that matters.
- **PHI handling:** Your prototype will touch data that resembles PHI: patient names, dates of birth, coverage and plan details, and Letters of Medical Necessity. **Use realistic mock/synthetic data only — never real patient data, and never real data pulled from any other source.** In both your build and your README, show us how you handle and protect that data: what you store, where, who can see it, and what you deliberately keep out.

- **AI guardrails (required if your prototype uses an LLM):** If your solution uses AI (for example, an assistant that answers "Is this covered in Florida?"), you must address safety directly. What happens when the model doesn't know the answer? How do you keep it from inventing coverage rules, pricing, or medical claims? When it's wrong, how does it fail safe? Cover this in your README and reflect it in the code. (Part 3 asks you to reason about this at a design level; here we want to see it reflected in what you actually built.)

What we're evaluating:

- **Product judgment** — Did you identify a meaningful problem and choose a high-impact solution?
- **User empathy** — Does your prototype reflect an understanding of what Sarah actually needs?
- **Technical execution** — Does the prototype work? Is the code clean, well-structured, and sensibly organized? Do your tests protect what matters?
- **Data responsibility** — Did you handle PHI-like data thoughtfully, and apply appropriate AI guardrails where relevant?
- **Communication** — Can you clearly explain what you built and why?

Part 3: Systems Integration (~30–45 minutes)

Now that you've built your prototype, take a step back and think about how it would fit into a real company's operations. Assume Cubby uses tools like Salesforce (CRM), Shopify (e-commerce), Kustomer (CX tool), and a content management system to run its business.

Answer the following in a few sentences each:

- **What internal systems would this need to connect to?** Where does the data come from, and where does it go? What integrations would you prioritize first?
- **How would your solution stay current?** If Cubby launches a new product, updates pricing, or changes a form, how does your prototype account for that? What breaks if nobody maintains it?
- **How would you keep coverage information correct?** Coverage and Medicaid rules vary by state and payer, and they change. When a rule changes, where does that update originate, and how does your prototype get the current version to Sarah? What's the source of truth? How do you keep cached or indexed data from going stale? And if that sync fails partway through, does Sarah see a wrong answer, a stale one, or none? Walk us through your design.

- **AI safety (everyone answers, even if your prototype uses no AI):** Imagine Cubby adds an AI assistant to the portal that answers partner questions like "Is the Cubby bed covered for a 7-year-old with autism in Florida?" How would you keep it from inventing coverage rules, pricing, or medical claims? What does it do when it isn't confident, and how does a wrong answer fail safe rather than mislead Sarah into a bad submission? If you built an AI component in Part 2, describe the guardrails you actually implemented and where they'd fall short at scale. If you didn't use AI, describe how you'd design these guardrails.
- **How would you measure whether this is actually working?** What signals would tell you that DME partners are getting value from your solution?

Submission Details

Prep call (optional): Before you begin, you're welcome to schedule a 20–30 minute call with Caleb Polley to walk through the assignment, ask clarifying questions, and get a feel for how we'd collaborate. Email Caleb@cubbybeds.com to request a call.

Expected time commitment: Approximately 4–6 hours.

Deadline: Please submit by the due date listed in the original email.

What to submit:

- Your written pitch (Part 1)
- Your prototype as a **GitHub repository link** with commit history (Part 2), including a README with setup instructions, your PHI-handling approach, and your AI guardrails (if applicable)
- Your systems integration responses (Part 3)

In your submission, include a brief note on what tools you used to build this (AI-assisted or otherwise), what stack you chose and why, and how it affected your output. We're genuinely curious about how you work.

How to submit: 'Reply all' to the email from Dustin & Caleb with your deliverables attached or linked.

Questions? If anything is unclear, please reach out to Caleb Polley. We're happy to clarify.

As a token of appreciation for your time, you will receive a \$400 Visa gift card upon submitting your case study.

A Few Notes

This is not production-ready work. The assignment is designed around the publicly available Supplier Portal and asks for a theoretical prototype, not a production deliverable we would ship as-is. It's meant to help us see how you think and build, and to give you a feel for the kind of work you'd do at Cubby.

If you proceed to the final interview round, expect to review and discuss this assignment and the approach you took. We'll ask follow-up questions about your choices, tradeoffs, and how you'd iterate on your solution.

A Note on AI Usage

Use AI however you want. We expect strong candidates to use it actively, for research, drafting, scaffolding, and sense-checking their thinking, and this is an AI-native role, so we want to see how you work with these tools. What we're evaluating is the quality of your judgment and the code you ship, not whether you typed every character yourself. The tasks are designed so that generic AI output that hasn't been thought through by a human is obvious: they require specific choices, real tradeoffs, guardrails you've actually reasoned about, and code that holds up when we read the repo. Tell us in your README how you used AI and where your own judgment changed what it gave you.